DOTE 6635: Artificial Intelligence for Business Research (Spring 2026)

Deep Reinforcement Learning

Renyu (Philip) Zhang

1

Where Are We?

- Model-based methods
  - We know the action space, the state space, the transition probability matrix, and the reward function.
  - Bellman equation to evaluate a policy and Bellman optimality operator to find the optimal policy.
  - Value iteration, policy iteration, linear programming.
- Model-free methods
  - Monte Carlo (MC) learns directly from episodes of experience: Sample the entire reward process and use empirical mean return to approximate expected return.
  - Temporal Difference (TD) combines MC and Bellman operator: Bootstrap to update the value function based on the existing estimate.
  - MC has zero bias and high variance; TD has some bias and low variance.
- Model-free control
  - MC policy iteration combines MC policy evaluation and ε –greedy for policy improvement.
  - SARSA is an on-policy method to update Q(S,A) using TD and ε –greedy for policy improvement.
  - Q-learning is an off-policy method to learn Q(S,A) for the greedy target policy using ε –greedy as the behavior policy through TD update.

2

2

Agenda

- **Value Function Approximation**
- DQN

3

3

Value Function Approximation

https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-6-value-function-approximation-.pdf

- So far we assume that we can represent the MDP using state and action pairs (Q-function).
  - This is called tabular RL.
- What if the state or action spaces are too large?
  - Continuous states: self-driving car
  - Large states: Go has $3^{19*19} \approx 10^{137}$ states (# of atoms in universe: ~10^{80})
  - Continuous actions: prices
- We try to scale up the model-free methods using approximations for value/ policy functions:

$$\hat{v}(s, \mathbf{w}) \approx v_\pi(s)$$

$$\text{or } \hat{q}(s, a, \mathbf{w}) \approx q_\pi(s, a)$$

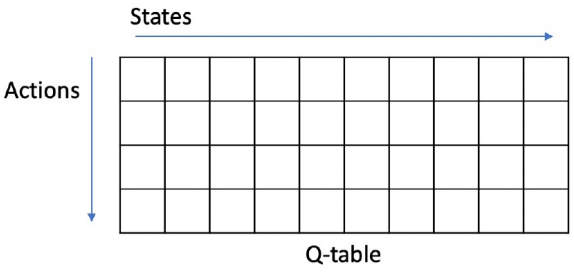
4

4

Value Function Approximation

https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-6-value-function-approximation-.pdf

- In tabular RL, we represent the value functions by a lookup table:
  - Every state s has an entry $V(s)$
  - Every state-action pair (s,a) has an entry $Q(s,a)$



- Challenges with large MDPs:
  - Too many states or actions to store in memory
  - Too slow to learn the value of each state individually

5

5

Value Function Approximation

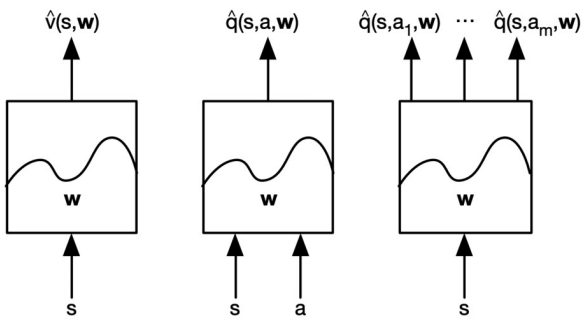
https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-6-value-function-approximation-.pdf

- Avoid explicitly learning or storing for every single state
  - Dynamics and reward model
  - Value function $V(s)$ and state-action function $Q(s,a)$
  - Policy
- Solution: Estimate approximated value and policy functions:

$$\hat{v}(s,\mathbf{w}) \approx v^{\pi}(s)$$
$$\hat{q}(s,a,\mathbf{w}) \approx q^{\pi}(s,a)$$
$$\hat{\pi}(a,s,\mathbf{w}) \approx \pi(a|s)$$

  - Generalize from seen states to unseen states
  - Update the parameter w using MC or TD learning.

Several Function Designs



6

6

Value Function Approximation

https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-6-value-function-approximation-.pdf

- Consider differentiable functions as the approximators that can deal with non-stationary and non-iid data:
  - Linear feature representations
  - Neural Nets
- Idea: Use gradient descent to update the value or policy approximators every time we see a sample.
- Value function approximation with an oracle.
  - We have the oracle for knowing the trye value for $v^{\pi}(s)$ for any given state s.
  - The objective is to find the best approximate representation of $v^{\pi}(s)$.
  - Use the MSE with loss function as:

$$J(\mathbf{w}) = \mathbb{E}_{\pi}\left[\left(v^{\pi}(s) - \hat{v}(s, \mathbf{w})\right)^2\right]$$

  - Then gradient descent:

$$\Delta \mathbf{w} = -\frac{1}{2}\alpha \nabla_{\mathbf{w}} J(\mathbf{w})$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \Delta \mathbf{w}$$

7

7

Linear Approximation with an Oracle

https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-6-value-function-approximation-.pdf

- Represent states using a finite vector with n variables: $x(s) = \begin{pmatrix} x_1(s) \\ x_2(s) \\ \cdots \\ x_n(s) \end{pmatrix}$
- Represent value function by a linear combination of features: $\hat{V}(s; \boldsymbol{w}) = \sum_{j=1}^{n} x_j(s) w_j = \boldsymbol{x}(s)^T \boldsymbol{w}$
- Loss function: $J(\boldsymbol{w}) = \mathbb{E}_{\pi}[(V^{\pi}(s) - \hat{V}(s; \boldsymbol{w}))^2]$
- Stochastic gradient descent to update the weights: $\Delta \boldsymbol{w} = -\frac{1}{2}\alpha \nabla_{\boldsymbol{w}} J(\boldsymbol{w})$
  - Since the function is linear and the loss is MSE:

$$\Delta \mathbf{w} = \alpha(v^{\pi}(s) - \hat{v}(s, \mathbf{w}))\mathbf{x}(s)$$

  - The loss is convex in w, so SGD will converge to the global minimum.
  - With a general approximation function (SGD will only converge to the local minimum):

$$\Delta \mathbf{w} = \alpha\left(v^{\pi}(s) - \hat{v}(s, \mathbf{w})\right)\nabla_{\mathbf{w}} \hat{v}(s_t, \mathbf{w})$$

8

8

Model Free Evaluation with Approximation

https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-6-value-function-approximation-.pdf

- In practice, we do not have oracles, just rewards collected by interacting with the environment.
- But we do obtain the target using samples.
- For MC, the target is simply the actual return G_t:

$$\Delta \mathbf{w} = \alpha\Big(G_t - \hat{v}(s_t, \mathbf{w})\Big)\nabla_{\mathbf{w}}\hat{v}(s_t, \mathbf{w})$$

  - G_t is an unbiased estimate of the oracle $v^{\pi}(s)$
  - MC prediction converges, in both linear and non-linear value function approximation.

Called semi-gradient, as we ignore the effect of changing w on the target.

- For TD(0), the target is the TD target $R_{t+1} + \gamma\hat{v}(s_{t+1}, \boldsymbol{w})$:

$$\Delta \mathbf{w} = \alpha\Big(R_{t+1} + \gamma\hat{v}(s_{t+1}, \mathbf{w}) - \hat{v}(s_t, \mathbf{w})\Big)\nabla_{\mathbf{w}}\hat{v}(s_t, \mathbf{w})$$

  - TD target $R_{t+1} + \gamma\hat{v}(s_{t+1}, \boldsymbol{w})$ is a biased estimate of the oracle $v^{\pi}(s)$
  - Linear TD(0) converges (close) to global minimum.

9

9

Algorithms

https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-6-value-function-approximation-.pdf

MC + SGD

while (not converged)

  $s_0 \sim p_0,\ t = 0$
  while ($s_t \neq$ <term>)
    $a_t \sim \pi(\cdot \mid s_t)$
    $(r_t, s_{t+1}) \sim p(\cdot, \cdot \mid s_t, a_t)$
    $t = t + 1$
  end
  $T = t$

} sample trajectory τ

  $g = \left(V_\phi(s_0) - \sum_{t=0}^{T-1} \gamma^t r_t\right)\nabla_\phi V_\phi(s_0)$

  update ϕ using g with an optimizer

end

TD + (semi-)SGD

while (not converged)

  $s_0 \sim p_0,\ a_0 \sim \pi(\cdot \mid s_0),\ (r_0, s_1) \sim p(\cdot, \cdot \mid s_0, a_0)$

  $y = \begin{cases} r_0 + \gamma V_\phi(s_1) & \text{if } s_1 \text{ != <term>} \\ r_0 & \text{if } s_1 \text{ == <term>} \end{cases}$

  $g = (V_\phi(s_0) - y)\nabla_\phi V_\phi(s_0)$

  update ϕ using g with an optimizer

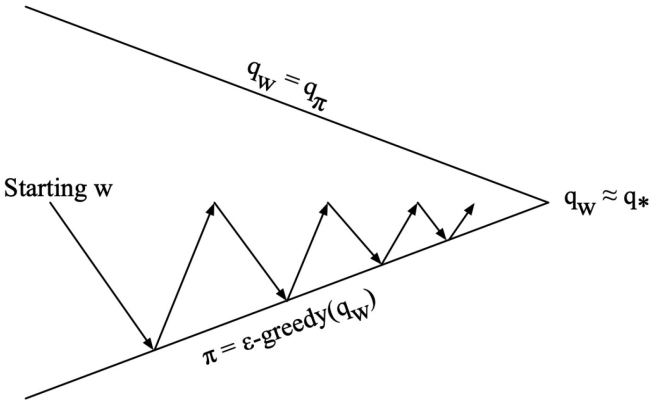
end

10

10

Generalized Policy Iteration with Value Function Approximation

https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-6-value-function-approximation-.pdf



1. Policy evaluation: approximate policy evaluation, $\hat{q}(.,.,\mathbf{w}) \approx q^\pi$
2. Policy improvement: ϵ-greedy policy improvement

11

11

Model-free Control with Linear Approximation with an Oracle

https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-6-value-function-approximation-.pdf

- With a similar idea, represent states using a finite vector with n variables: $\boldsymbol{x}(s,a) = \begin{pmatrix} x_1(s,a) \\ x_2(s,a) \\ \cdots \\ x_n(s,a) \end{pmatrix}$
- Use a linear combination of features to approximate Q-function so as to optimize over actions:

$$\hat{Q}(s,a;\boldsymbol{w}) = \boldsymbol{x}(s,a)^T \boldsymbol{w} = \sum_{j=1}^{n} x_j(s,a) w_j$$

- Minimize the MSE between the approximate Q-function and the oracle Q-function.

$$J(\mathbf{w}) = \mathbb{E}_\pi[(q^\pi(s,a) - \hat{q}(s,a,\mathbf{w}))^2]$$

- Stochastic gradient descent update:

$$\nabla_{\boldsymbol{w}} J(\boldsymbol{w}) = \nabla_{\boldsymbol{w}} \mathbb{E}_\pi[(Q^\pi(s,a) - \hat{Q}^\pi(s,a;\boldsymbol{w}))^2]$$

$$\Delta\mathbf{w} = \alpha(q^\pi(s,a) - \hat{q}(s,a,\mathbf{w}))\nabla_{\mathbf{w}}\hat{q}(s,a,\mathbf{w}) \qquad \Delta\mathbf{w} = \alpha(q^\pi(s,a) - \hat{q}(s,a,\mathbf{w}))\mathbf{x}(s,a)$$

12

12

Incremental Control Algorithms

https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-6-value-function-approximation-.pdf

- In practice, we do not have oracle state-action function $q^{\pi}(s,a)$, just rewards collected by interacting with the environment.
- For MC, the target is simply the actual return G_t:

$$\Delta \mathbf{w} = \alpha\big(G_t - \hat{q}(s_t, a_t, \mathbf{w})\big)\nabla_{\mathbf{w}}\hat{q}(s_t, a_t, \mathbf{w})$$

- For SARSA, the target is the TD target $R_{t+1} + \gamma\hat{q}(s_{t+1}, a_{t+1}, \boldsymbol{w})$:

$$\Delta \mathbf{w} = \alpha\big(R_{t+1} + \gamma\hat{q}(s_{t+1}, a_{t+1}, \mathbf{w}) - \hat{q}(s_t, a_t, \mathbf{w})\big)\nabla_{\mathbf{w}}\hat{q}(s_t, a_t, \mathbf{w})$$

- For Q-learning, the target is TD target $R_{t+1} + \gamma \max_a \gamma\hat{q}(s_{t+1}, a, \boldsymbol{w})$:

$$\Delta \mathbf{w} = \alpha\big(R_{t+1} + \gamma \max_a \hat{q}(s_{t+1}, a, \mathbf{w}) - \hat{q}(s_t, a_t, \mathbf{w})\big)\nabla_{\mathbf{w}}\hat{q}(s_t, a_t, \mathbf{w})$$

13

13

Algorithms

https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-6-value-function-approximation-.pdf

MC + SGD

while (not converged)

$s_0 \sim p_0,\ a_0 \sim \pi(\cdot \mid s_0),\ t = 0$
while ($s_t \neq$ <term>)
$a_t \sim \pi(\cdot \mid s_t)$
$(r_t, s_{t+1}) \sim p(\cdot, \cdot \mid s_t, a_t)$
$t = t + 1$
end
$T = t$

} sample trajectory τ

$$g = \left(Q_\phi(s_0, a_0) - \sum_{t=0}^{T-1} \gamma^t r_t\right)\nabla_\phi Q_\phi(s_0, a_0)$$

update ϕ using g with an optimizer

end

TD + (semi-)SGD

Episodic Semi-gradient Sarsa for Estimating $\hat{q} \approx q_*$

Input: a differentiable function $\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \to \mathbb{R}$

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)
Repeat (for each episode):
  $S, A \leftarrow$ initial state and action of episode (e.g., ε-greedy)
  Repeat (for each step of episode):
    Take action A, observe R, S'
    If S' is terminal:
      $\mathbf{w} \leftarrow \mathbf{w} + \alpha[R - \hat{q}(S, A, \mathbf{w})]\nabla\hat{q}(S, A, \mathbf{w})$
      Go to next episode
    Choose A' as a function of $\hat{q}(S', \cdot, \mathbf{w})$ (e.g., ε-greedy)
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha[R + \gamma\hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})]\nabla\hat{q}(S, A, \mathbf{w})$
    $S \leftarrow S'$
    $A \leftarrow A'$

14

14

Convergence of Control Algorithms

https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-6-value-function-approximation-.pdf

- SARSA: $\Delta \mathbf{w} = \alpha\big(R_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, \mathbf{w}) - \hat{q}(s_t, a_t, \mathbf{w})\big)\nabla_{\mathbf{w}} \hat{q}(s_t, a_t, \mathbf{w})$
- Q-learning: $\Delta \mathbf{w} = \alpha\big(R_{t+1} + \gamma \max_a \hat{q}(s_{t+1}, a, \mathbf{w}) - \hat{q}(s_t, a_t, \mathbf{w})\big)\nabla_{\mathbf{w}} \hat{q}(s_t, a_t, \mathbf{w})$
- TD w. value function approximation does NOT follow the gradient of any loss function.
- The updates essentially involves approximate Bellman backup + fitting the underlying value function.
- TD may diverge in off-policy (e.g., Q-learning) or non-linear approximation settings.
- Challenge for off-policy control: Behavior policy and target policy are not identical, so value function approximation may diverge.

Convergence of Approximate Model-free Control

Algorithm	Table Lookup	Linear	Non-Linear
Monte-Carlo Control	√	(√)	X
Sarsa	√	(√)	X
Q-Learning	√	X	X

(√) moves around the near-optimal value function

Deadly Triad:
- Bootstrapping
- Function approximation
- Off-policy learning

15

15

Agenda

- Value Function Approximation
- **DQN**

16

16

Going Beyond Linear

https://web.stanford.edu/class/cs234/slides/lecture4pre.pdf

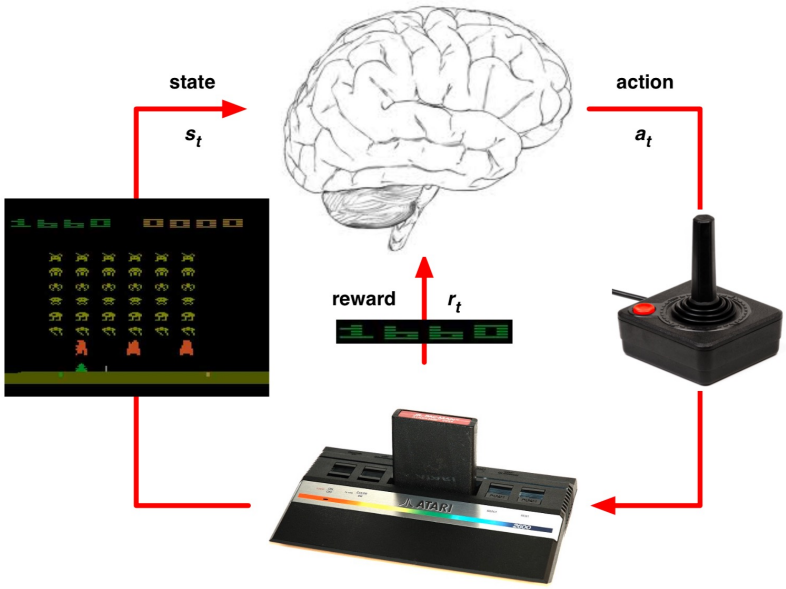
- So far, we have focused on linear approximators.
  - Linear approximations work well given the right set of features.
  - It requires manual design of the feature set.
- We know that DNN is a much better representation tool if we have a large data set.
- Question: How can we leverage deep learning for function approximation and model-free control of MDPs?

17

17

Deep Reinforcement Learning in Atari

https://web.stanford.edu/class/cs234/slides/lecture4pre.pdf



18

18

Deep Q-Network (DQN) for Atari

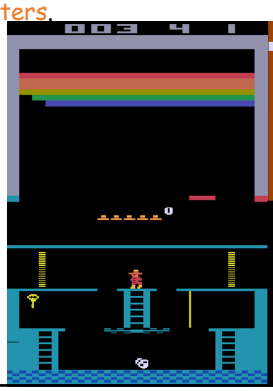
https://web.stanford.edu/class/cs234/slides/lecture4pre.pdf

- Groud-breaking paper by DeepMind:

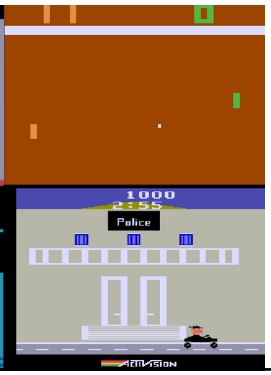
Human-level control through deep reinforcement learning
V Mnih, K Kavukcuoglu, D Silver, AA Rusu, J Veness… - nature, 2015 - nature.com
… large neural networks using a reinforcement learning signal and stochastic gradient descent in a stable manner— illustrated by the temporal evolution of two indices of learning (the …
☆ 保存 99 引用 被引用次数：40192 相关文章 所有 55 个版本

- DQN represents the action-value function with DNN approximator.
- DQN reaches a professional human gaming level across many Atari games using the same network and hyperparameters.

Breakout



Pong



Montezuma's Revenge

Private Eye

Game	Linear	Deep Network
Breakout	3	3
Enduro	62	29
River Raid	2345	1453
Seaquest	656	275
Space Invaders	301	302

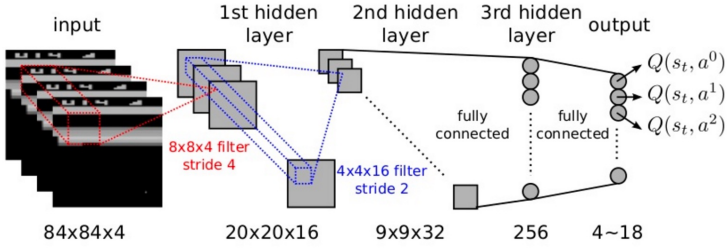
19

19

Deep Q-Network (DQN) for Atari

https://web.stanford.edu/class/cs234/slides/lecture4pre.pdf

- End-to-end learning of $Q(s,a)$ from input fixel frame.
- Input state s is a stack of raw pixels from the latest 4 frames.
- Output of $Q(s,a)$ is 18 buttons.
- Reward is the change in score for that step.
- Network arthitecture and hyperparameters fixed across all games:



Abstract

The theory of reinforcement learning provides a normative account[1], deeply rooted in psychological[2] and neuroscientific[3] perspectives on animal behaviour, of how agents may optimize their control of an environment. To use reinforcement learning successfully in situations approaching real-world complexity, however, agents are confronted with a difficult task: they must derive efficient representations of the environment from high-dimensional sensory inputs, and use these to generalize past experience to new situations. Remarkably, humans and other animals seem to solve this problem through a harmonious combination of reinforcement learning and hierarchical sensory processing systems[4,5], the former evidenced by a wealth of neural data revealing notable parallels between the phasic signals emitted by dopaminergic neurons and temporal difference reinforcement learning algorithms[3]. While reinforcement learning agents have achieved some successes in a variety of domains[6,7,8], their applicability has previously been limited to domains in which useful features can be handcrafted, or to domains with fully observed, low-dimensional state spaces. Here we use recent advances in training deep neural networks[9,10,11] to develop a novel artificial agent, termed a deep Q-network, that can learn successful policies directly from high-dimensional sensory inputs using end-to-end reinforcement learning. We tested this agent on the challenging domain of classic Atari 2600 games[12]. We demonstrate that the deep Q-network agent, receiving only the pixels and the game score as inputs, was able to surpass the performance of all previous algorithms and achieve a level comparable to that of a professional human games tester across a set of 49 games, using the same algorithm, network architecture and hyperparameters. This work bridges the divide between high-dimensional sensory inputs and actions, resulting in the first artificial agent that is capable of learning to excel at a diverse array of challenging tasks.

20

20

Deep Reinforcement Learning

https://web.stanford.edu/class/cs234/slides/lecture4pre.pdf

- We leverage DNNs to represent:
  - Value functions (Q & V)
  - Policy functions (π)
  - World model
- Loss function is optimized by stochastic gradident descent (SGD)
- Core challenges:
  - Data inefficiency: Too many model parameters to optimize.
  - Deadly Triad creates instability and divergence in training:
    - Nonlinear function approximation
    - Bootstrapping
    - Off-policy training
- Deep Q-learning (DQN) addresses correlations between samples and non-stationary targets through:
  - Experience replay: Sample experience randomly from data to update weights, reducing correlations between data points.
  - Fixed Q-targets: Fix the target networks' weights while updating, improving stability.

21

21

DQN: Experience Replay

https://web.stanford.edu/class/cs234/slides/lecture4pre.pdf

- To reduce the correlations among samples, store the transition (s_t, a_t, r_t, s_{t+1}) in replay memory D.

s_1, a_1, r_2, s_2
s_2, a_2, r_3, s_3
s_3, a_3, r_4, s_4
...
$s_t, a_t, r_{t+1}, s_{t+1}$

$\rightarrow$ s, a, r, s'

- To perform experience replay, repeat the following:
  1. Sample an experience tuple from the dataset: $(s, a, r, s') \sim D$.
  2. Compute the target value for the sampled tuple: $r + \gamma \max_{a'} \hat{Q}(s', a', \mathbf{w})$
  3. Use SGD to update the network weights w:

$$\Delta \mathbf{w} = \alpha\left(r + \gamma \max_{a'} \hat{Q}(s', a', \mathbf{w}) - Q(s, a, \mathbf{w})\right) \nabla_{\mathbf{w}} \hat{Q}(s, a, \mathbf{w})$$

- Use target as a scaler, but network weights will get updated on the next round, changing the target value.

22

22

DQN: Fixed Targets

https://web.stanford.edu/class/cs234/slides/lecture4pre.pdf

- To improve stability, fixe the target weights used in the target calculation for multiple updates.
- Target network uses a different set of weights than the weights being updated.
- Let a different set of parameter $\mathbf{w}^-$, be the set of weights used in the target, and $\mathbf{w}$ be the weights that are being updated.
- To perform experience replay with fixed target, repeat the following:
  1. Sample an experience tuple from the dataset: $(s, a, r, s') \sim D$
  2. Compute the target value for the sampled tuple: $r + \gamma \max_{a'} \hat{Q}(s', a', \mathbf{w}^-)$
  3. Use stochastic gradient descent to update the network weights:

$$\Delta \mathbf{w} = \alpha\left(r + \gamma \max_{a'} \hat{Q}(s', a', \mathbf{w}^-) - Q(s, a, \mathbf{w})\right)\nabla_{\mathbf{w}} \hat{Q}(s, a, \mathbf{w})$$

23

23

DQN Algorithm

https://web.stanford.edu/class/cs234/slides/lecture4pre.pdf

1: Input C, α, $D = \{\}$, Initialize $\boldsymbol{w}$, $\boldsymbol{w}^- = \boldsymbol{w}$, $t = 0$
2: Get initial state s_0
3: **loop**
4: Sample action a_t given ϵ-greedy policy for current $\hat{Q}(s_t, a; \boldsymbol{w})$
5: Observe reward r_t and next state s_{t+1}
6: Store transition (s_t, a_t, r_t, s_{t+1}) in replay buffer D
7: Sample random minibatch of tuples (s_i, a_i, r_i, s_{i+1}) from D
8: **for** j in minibatch **do**
9: **if** episode terminated at step $i + 1$ **then**
10: $y_i = r_i$
11: **else**
12: $y_i = r_i + \gamma \max_{a'} \hat{Q}(s_{i+1}, a'; \boldsymbol{w}^-)$
13: **end if**
14: Do gradient descent step on $(y_i - \hat{Q}(s_i, a_i; \boldsymbol{w}))^2$ for parameters $\boldsymbol{w}$: $\Delta \boldsymbol{w} = \alpha(y_i - \hat{Q}(s_i, a_i; \boldsymbol{w}))\nabla_{\boldsymbol{w}} \hat{Q}(s_i, a_i; \boldsymbol{w})$
15: **end for**
16: $t = t + 1$
17: **if** mod(t,C) == 0 **then**
18: $\boldsymbol{w}^- \leftarrow \boldsymbol{w}$
19: **end if**
20: **end loop**

Sample code:

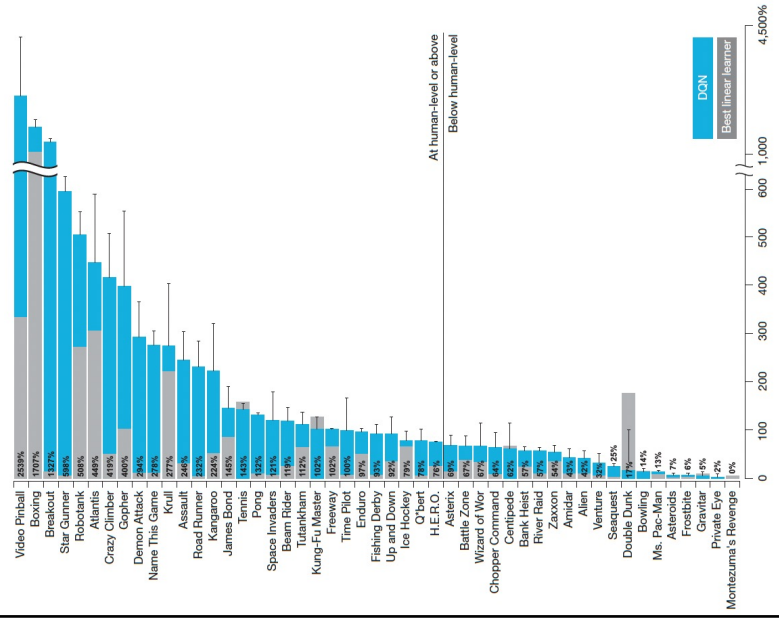
- https://www.tensorflow.org/agents/tutorials/0_intro_rl
- https://keras.io/examples/rl/deep_q_network_breakout/

24

24

Performance of DQN on Atari

https://web.stanford.edu/class/cs234/slides/lecture4pre.pdf



25

25

Performance of DQN on Atari

https://web.stanford.edu/class/cs234/slides/lecture4pre.pdf

	Replay Fixed-Q	Replay Q-learning	No replay Fixed-Q	No replay Q-learning
Breakout	316.81	240.73	10.16	3.17
Enduro	1006.3	831.25	141.89	29.1
River Raid	7446.62	4102.81	2867.66	1453.02
Seaquest	2894.4	822.55	1003	275.81
Space Invaders	1088.94	826.33	373.22	301.99

26

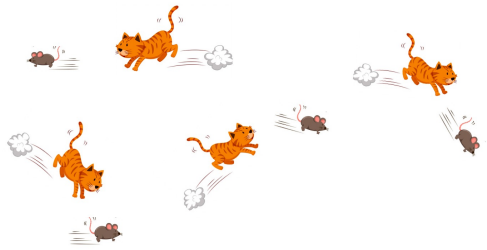
26

Why Fixed Targets?

https://web.stanford.edu/class/cs234/slides/lecture4pre.pdf

- In the original Q-learning with value function approximation, both Q estimation and Q target shifts at each time step.
- Think of a cat (Q estimation) is chasing a mouse (Q target), with the purpose that the cat reduces the distance to the mouse.

- If both the cat and the mouse are moving, we will observe an oscillated training history; so, we fix the target for a period of time during training.



27

27

Extensions to DQN

https://web.stanford.edu/class/cs234/slides/lecture4pre.pdf

- The success of DQN on Atari inspires a set of papers improving deep reinforcement learning.
  - Double DQN (AAAI 2016), Prioritized Replay (ICLR 2016), Dueling DQN (best paper at ICML 2016)
  - Also, huge economic values through self-driving cars.

Deep reinforcement learning with double q-learning
H Van Hasselt, A Guez, D Silver - … of the AAAI conference on artificial …, 2016 - ojs.aaai.org
… The popular Q-learning algorithm is known to overestimate … algorithm, which combines Q-learning with a deep neural network, … We then show that the idea behind the Double Q-learning …
☆ 保存 99 引用 被引用次数：12768 相关文章 所有 15 个版本 »

Prioritized experience replay
T Schaul, J Quan, I Antonoglou, D Silver - arXiv preprint arXiv:1511.05952, 2015 - arxiv.org
… To demonstrate the potential effectiveness of prioritizing replay by TD … prioritization' algorithm. This algorithm stores the last encountered TD error along with each transition in the replay …
☆ 保存 99 引用 被引用次数：7115 相关文章 所有 12 个版本 »

Dueling network architectures for deep reinforcement learning
Z Wang, T Schaul, M Hessel… - … machine learning, 2016 - proceedings.mlr.press
… Still, many of these applications use conventional architectures, such as convolutional … neural network architecture for model-free reinforcement learning. Our dueling network represents …
☆ 保存 99 引用 被引用次数：6631 相关文章 所有 8 个版本 »

28

28

Deep Q-Learning for Dynamic Coupon Targeting

informs
https://pubsonline.informs.org/journal/mksc

MARKETING SCIENCE
Articles in Advance, pp. 1–22
ISSN 0732-2399 (print), ISSN 1526-548X (online)

Dynamic Coupon Targeting Using Batch Deep Reinforcement Learning: An Application to Livestream Shopping

Xiao Liu[a]

[a] Stern School of Business, New York University, New York, New York 10012
Contact: xl23@stern.nyu.edu, https://orcid.org/0000-0002-7093-8534 (XL)

Received: September 3, 2020
Revised: September 5, 2021; April 17, 2022
Accepted: June 23, 2022
Published Online in Articles in Advance: October 20, 2022

https://doi.org/10.1287/mksc.2022.1403

Abstract. We present an empirical framework for creating dynamic coupon targeting strategies for high-dimensional and high-frequency settings, and we test its performance using a large-scale field experiment. The framework captures consumers' intertemporal tradeoffs associated with dynamic pricing and does not rely on functional form assumptions about consumers' decision-making processes. The model is estimated using batch deep reinforcement learning (BDRL), which relies on Q-learning, a model-free solution that can mitigate model bias. It leverages deep neural networks to represent the high-dimensional state space and alleviate the curse of dimensionality. The empirical application is in a multibillion-dollar livestream shopping context. Our BDRL solution increases the platform's revenue by twice as much as static targeting policies and by 20% more than the model-based solution. The comparative advantage of BDRL comes from more effective and automatic targeting of consumers based on both heterogeneity and dynamics, using exceptionally rich, nuanced differences among consumers and across time. We find that price skimming, reducing discounts for attractive hosts, and increasing the coupon discount level at a faster rate for low spenders are effective strategies based on dynamics, consumer heterogeneity, and the two combined, respectively.

History: K. Sudhir served as the senior editor and John Hauser served as associate editor for this article.
Funding: Partial financial support was received from the NYU Center for Global Economy and Business.
Supplemental Material: The data files and online appendices are available at https://doi.org/10.1287/mksc.2022.1403.

Keywords: dynamic pricing • coupon • deep reinforcement learning • reference price • livestream shopping • targeting

Dynamic coupon targeting using batch deep reinforcement learning: An application to livestream shopping
X Liu - Marketing Science, 2023 - pubsonline.informs.org
… incorporates the intertemporal tradeoffs in dynamic coupon targeting to design a policy …
dynamic coupon targeting policies? (3) What are the gains, if any, from using a dynamic targeting …
☆ 保存 99 引用 被引用次数：115 相关文章 所有 7 个版本

Algorithm 2 (BCQ)

1. Input: Batch data $\mathcal{B}$, horizon T, target network update rate Υ, mini-batch size N, threshold τ.
2. Initialize Q-networks Q_θ, generative model G_ω (trained in a standard supervised learning fashion, with cross-entropy loss), and target networks $Q_{\theta'}$, with $\theta' \leftarrow \theta$.
3. for $t = 1$ to T, do:
4. Sample mini-batch M of N transitions $(\mathbf{S}, A, R, \mathbf{S}')$ from $\mathcal{B}$
5. $A' = \arg\max_{A' | \frac{G_\omega(A'|\mathbf{S}')}{\max_{\hat{A}} G_\omega(\hat{A}|\mathbf{S}')} > \tau} Q_\theta(\mathbf{S}', A')$
6. $\theta \leftarrow \arg\min_\theta \sum_{(\mathbf{S},A,R,\mathbf{S}') \in M} l_\kappa(R + \gamma Q_{\theta'}(\mathbf{S}', A') - Q_{\theta'}(\mathbf{S}, A))$
7. $\omega \leftarrow \arg\min_\omega - \sum_{(\mathbf{S},A) \in M} \log G_\omega(A \mid \mathbf{S})$
8. If $t \bmod \Upsilon = 0$: $\theta' \leftarrow \theta$
9. end for

Table 7. Model Comparison Based on the Doubly Robust Estimator

		1. Static policy with homogeneous treatments	2. Static policy with heterogeneous treatments			3. Model-based dynamic policy with heterogeneous treatments	4. Model-free dynamic policy with heterogeneous treatments
		A: Regression	B: GBDT	C: DNN	D: ORF	E: Structural[a]	F: Proposed BDRL
CLV (Return)	Mean	6.53	7.52	6.95	7.49	8.37	9.53
	Std	2.00	2.29	2.27	2.29	2.77	3.23
Gain	Mean	11%	28%	18%	28%	43%	62%
t test[b]		222.99	524.25	346.60	515.47	714.27	945.57
p value		<0.001	<0.001	<0.001	<0.001	<0.001	<0.001

Notes. All the gains are compared with the mean CLV of ¥5.87. The total sample size is 1,020,898 consumers.

29